# 预约挂号模块讲解文档

## 一、数据库与实体类设计

### 1.1 核心实体表设计

#### 1.1.1 预约实体表(appointment)

```
CREATE TABLE `appointment` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '预约ID',
  `patient_id` bigint NOT NULL COMMENT '患者ID',
  `doctor_id` bigint NOT NULL COMMENT '医生ID',
  `schedule_id` bigint NOT NULL COMMENT '排班ID',
  `appointment_no` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NOT NULL COMMENT '预约编号',
  `appointment_date` date NOT NULL COMMENT '预约日期',
  `time_slot` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT
NULL COMMENT '时间段(上午/下午/晚上)',
  `symptoms` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL COMMENT
'症状描述',
  `status` tinyint NULL DEFAULT 1 COMMENT '状态(0:取消,1:待就诊,2:已就诊)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `uk_appointment_no`(`appointment_no` ASC) USING BTREE,
  INDEX `idx_patient_id`(`patient_id` ASC) USING BTREE,
  INDEX `idx_doctor_id`(`doctor_id` ASC) USING BTREE,
  INDEX `idx_schedule_id`(`schedule_id` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '预约表' ROW_FORMAT = Dynamic;
```

#### 1.1.2 排班实体表(schedule)

```
CREATE TABLE `schedule` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '排班ID',
  `doctor_id` bigint NOT NULL COMMENT '医生ID',
  `schedule_date` date NOT NULL COMMENT '排班日期',
  `time_slot` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT
NULL COMMENT '时间段(上午/下午/晚上)',
  `max_patients` int NULL DEFAULT 0 COMMENT '最大接诊人数',
  `current_patients` int NULL DEFAULT 0 COMMENT '当前预约人数',
  `status` tinyint NULL DEFAULT 1 COMMENT '状态(0:停诊,1:正常)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`),
  INDEX `idx_doctor_date`(`doctor_id`, `schedule_date`) USING BTREE
)
```

半夜撕代码版权所有

## 1.2 核心实体类

### 1.2.1 Appointment实体类

```
@Data
@TableName("appointment")
@Schema(description = "预约实体类")
public class Appointment {
    @TableId(type = IdType.AUTO)
    @Schema(description = "预约ID")
    private Long id;

    @Schema(description = "患者ID")
    private Long patientId;

    @Schema(description = "医生ID")
    private Long doctorId;

    @Schema(description = "排班ID")
    private Long scheduleId;

    @Schema(description = "预约编号")
    private String appointmentNo;

    @Schema(description = "预约日期")
    private LocalDate appointmentDate;

    @Schema(description = "时间段(上午/下午/晚上)")
    private String timeSlot;

    @Schema(description = "症状描述")
    private String symptoms;

    @Schema(description = "状态(0:取消,1:待就诊,2:已就诊)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "患者信息")
    private Patient patient;

    @TableField(exist = false)
    @Schema(description = "医生信息")
    private Doctor doctor;

    @TableField(exist = false)
    @Schema(description = "排班信息")
    private Schedule schedule;
}
```

半夜撕代码版权所有

### 1.2.2 Schedule实体类

```
@Data
@TableName("schedule")
@Schema(description = "排班实体类")
public class Schedule {
    @TableId(type = IdType.AUTO)
    @Schema(description = "排班ID")
    private Long id;

    @Schema(description = "医生ID")
    private Long doctorId;

    @Schema(description = "排班日期")
    private LocalDate scheduleDate;

    @Schema(description = "时间段(上午/下午/晚上)")
    private String timeSlot;

    @Schema(description = "最大接诊人数")
    private Integer maxPatients;

    @Schema(description = "当前预约人数")
    private Integer currentPatients;

    @Schema(description = "状态(0:停诊,1:正常)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "医生信息")
    private Doctor doctor;

    @TableField(exist = false)
    @Schema(description = "剩余可预约数")
    private Integer remainingQuota;
}
```

## 1.3 核心属性说明

1. **预约状态**：使用整数表示不同状态，0表示已取消，1表示待就诊，2表示已就诊
2. **排班状态**：使用整数表示不同状态，0表示停诊，1表示正常
3. **时间段**：使用字符串表示，包括"上午"、"下午"、"晚上"
4. **预约编号**：系统自动生成的唯一编号，用于标识预约记录
5. **剩余可预约数**：通过计算属性获取，等于最大接诊人数减去当前预约人数

半夜撕代码版权所有

# 二、各功能详细讲解

## 2.1 患者预约挂号功能

### 2.1.1 功能概述

患者预约挂号功能允许患者通过系统选择科室、医生和时间段进行挂号预约。整个流程包括选择科室、选择医生、查看医生排班、填写症状描述等步骤，确保患者能够方便地完成预约挂号流程。

### 2.1.2 详细讲解功能实现流程

1. **前端页面加载**：
   - 用户访问预约挂号页面，系统首先检查用户是否已登录
   - 如未登录，则提示用户登录并跳转到登录页面
   - 如已登录，则加载科室列表
2. **科室选择**：
   - 前端调用 `fetchDepartments` 方法，向后端发送GET请求获取科室列表
   - 用户点击科室，触发 `handleDepartmentChange` 方法，更新选中的科室ID
   - 系统调用 `fetchDoctorsByDepartment` 方法获取该科室下的医生列表
3. **医生选择**：
   - 用户选择医生，触发 `handleDoctorSelect` 方法，更新选中的医生ID
   - 系统自动设置默认日期范围为今天开始的7天
   - 调用 `fetchDoctorSchedules` 方法获取医生排班信息
4. **时间选择**：
   - 用户可以修改日期范围，触发 `handleDateRangeChange` 方法
   - 系统重新获取医生在选定日期范围内的排班信息
   - 前端展示医生排班列表，包括日期、时间段和剩余可预约数
5. **提交预约**：
   - 用户点击"预约"按钮，触发 `handleAppointment` 方法
   - 系统检查用户是否已完善患者信息，如未完善则提示用户先完善信息
   - 弹出预约确认对话框，用户填写症状描述
   - 用户点击"确认预约"，触发 `submitAppointment` 方法
   - 前端向后端发送POST请求创建预约记录
   - 后端进行预约验证，包括检查患者是否存在、医生是否存在、排班是否存在、是否有剩余名额等
   - 成功后更新排班的当前预约人数，并返回预约结果

### 2.1.3 关键代码讲解

1. **前端预约挂号页面结构**
   【vue3/src/views/frontend/appointment/index.vue】

```
<el-row :gutter="20">
  <!-- 左侧科室和医生选择 -->
```

半夜撕代码版权所有

```
  <el-col :md="8" :sm="24">
    <el-card class="selection-card" shadow="hover">
      <!-- 科室列表 -->
      <div class="department-list">
        <el-scrollbar height="180px">
          <div class="dept-buttons">
            <el-button
              v-for="dept in departmentList"
              :key="dept.id"
              :class="['dept-button', selectedDepartmentId === dept.id ?
'dept-active' : '']"
              @click="handleDepartmentChange(dept.id)"
            >
              {{ dept.deptName }}
            </el-button>
          </div>
        </el-scrollbar>
      </div>

      <!-- 医生列表 -->
      <div class="doctor-list">
        <!-- 医生卡片 -->
      </div>
    </el-card>
  </el-col>

  <!-- 右侧排班和预约 -->
  <el-col :md="16" :sm="24">
    <!-- 排班日历 -->
    <div class="schedule-list">
      <div
        v-for="schedule in scheduleList"
        :key="schedule.id"
        class="schedule-item"
      >
        <!-- 排班信息 -->
        <el-button
          :disabled="schedule.maxPatients <= schedule.currentPatients"
          @click="handleAppointment(schedule)"
        >
          {{ schedule.maxPatients <= schedule.currentPatients ? '已满' : '预
约' }}
        </el-button>
      </div>
    </div>
  </el-col>
</el-row>
```

2. **前端获取医生排班方法**
【vue3/src/views/frontend/appointment/index.vue：fetchDoctorSchedules方法】

```
const fetchDoctorSchedules = async () => {
  if (!selectedDate.value || selectedDate.value.length !== 2) {
    return Promise.resolve()
  }
```

半夜撕代码版权所有

```
  try {
    const startDate = formatDateForApi(selectedDate.value[0])
    const endDate = formatDateForApi(selectedDate.value[1])

    return new Promise((resolve) => {
      request.get(`/schedule/doctor/${selectedDoctorId.value}`, {
        startDate,
        endDate
      }, {
        onSuccess: (res) => {
          scheduleList.value = res || []
          resolve()
        }
      })
    })
  } catch (error) {
    console.error('获取医生排班失败:', error)
    return Promise.reject(error)
  }
}
```

3. **前端提交预约方法**

【vue3/src/views/frontend/appointment/index.vue：submitAppointment方法】

```
const submitAppointment = () => {
  appointmentFormRef.value.validate(async (valid) => {
    if (valid) {
      try {
        await request.post('/appointment', appointmentForm, {
          successMsg: '预约成功',
          onSuccess: () => {
            appointmentDialogVisible.value = false

            // 刷新排班列表
            fetchDoctorSchedules()

            // 询问是否查看预约记录
            ElMessageBox.confirm(
              '预约成功，是否查看我的预约记录？',
              '预约成功',
              {
                confirmButtonText: '查看预约',
                cancelButtonText: '继续预约',
                type: 'success',
              }
            )
              .then(() => {
                router.push('/my-appointments')
              })
              .catch(() => {})
          }
        })
      } catch (error) {
        console.error('预约失败:', error)
      }
    }
```

半夜撕代码版权所有

```
  })
}
```

4. **后端创建预约方法**
【springboot/src/main/java/org/example/springboot/service/AppointmentService.java：createAppointment方法】

```
@Transactional
public Appointment createAppointment(Appointment appointment) {
    // 检查患者是否存在
    Patient patient = patientMapper.selectById(appointment.getPatientId());
    if (patient == null) {
        throw new ServiceException("患者不存在");
    }

    // 检查医生是否存在
    Doctor doctor = doctorMapper.selectById(appointment.getDoctorId());
    if (doctor == null) {
        throw new ServiceException("医生不存在");
    }

    // 检查排班是否存在
    Schedule schedule =
scheduleMapper.selectById(appointment.getScheduleId());
    if (schedule == null) {
        throw new ServiceException("排班不存在");
    }

    // 检查排班状态是否正常
    if (schedule.getStatus() != 1) {
        throw new ServiceException("该排班已停诊，无法预约");
    }

    // 检查是否有剩余预约名额
    if (schedule.getCurrentPatients() >= schedule.getMaxPatients()) {
        throw new ServiceException("该排班已满，无法预约");
    }

    // 检查患者是否已经预约过同一天同一医生的门诊
    LambdaQueryWrapper<Appointment> queryWrapper = new LambdaQueryWrapper<>
();
    queryWrapper.eq(Appointment::getPatientId, appointment.getPatientId())
                .eq(Appointment::getDoctorId, appointment.getDoctorId())
                .eq(Appointment::getAppointmentDate,
schedule.getScheduleDate())
                .eq(Appointment::getStatus, 1); // 待就诊状态

    if (appointmentMapper.selectCount(queryWrapper) > 0) {
        throw new ServiceException("您已预约过该医生当天的门诊");
    }

    // 生成预约编号
    appointment.setAppointmentNo(generateAppointmentNo());

    // 设置预约日期和时间段
    appointment.setAppointmentDate(schedule.getScheduleDate());
```

半夜撕代码版权所有

```
        appointment.setTimeSlot(schedule.getTimeSlot());

        // 设置状态为待就诊
        appointment.setStatus(1);

        // 设置创建时间和更新时间
        LocalDateTime now = LocalDateTime.now();
        appointment.setCreateTime(now);
        appointment.setUpdateTime(now);

        // 保存预约信息
        if (appointmentMapper.insert(appointment) <= 0) {
            throw new ServiceException("预约失败");
        }

        // 更新排班的当前预约人数
        scheduleService.updateSchedulePatients(schedule.getId(), 1);

        return appointment;
    }
```

### 2.1.4 功能实现细节

1. **预约编号生成**：
   系统自动生成唯一的预约编号，格式为"YY" + 当前年份后两位 + 月份两位 + 日期两位 + 6位随机数，确保每个预约都有唯一标识。
2. **预约冲突检查**：
   系统会检查患者是否已经预约过同一天同一医生的门诊，避免重复预约。
3. **排班名额管理**：
   每次成功预约后，系统会自动更新排班的当前预约人数，确保不会超出最大接诊人数。
4. **用户体验优化**：
   - 预约成功后，系统会询问用户是否查看预约记录
   - 如果用户未完善患者信息，系统会提示用户先完善信息再预约
   - 已满的排班会显示"已满"并禁用预约按钮

## 2.2 预约管理功能

### 2.2.1 功能概述

预约管理功能包括患者查看自己的预约记录、取消预约，以及医生和管理员查看和管理所有预约记录。系统支持按照多种条件筛选预约记录，并提供详细的预约信息查看功能。

### 2.2.2 详细讲解功能实现流程

1. **患者查看预约记录**：
   - 患者访问"我的预约"页面
   - 前端调用 `fetchAppointments` 方法，向后端发送GET请求获取当前患者的预约记录
   - 系统展示预约列表，包括预约状态、时间、医生信息等
2. **患者取消预约**：
   - 患者点击"取消预约"按钮，触发 `handleCancelAppointment` 方法

半夜撕代码版权所有

- 前端弹出确认对话框，用户确认后向后端发送PUT请求取消预约
- 后端验证预约状态，更新预约状态为已取消，并更新排班的当前预约人数

3. **医生/管理员查看预约记录**：
   - 医生/管理员访问预约管理页面
   - 前端根据用户角色自动设置查询条件（医生只能查看自己的预约）
   - 调用 `fetchAppointments` 方法，向后端发送GET请求获取预约记录
   - 系统展示预约列表，支持多条件筛选
4. **医生完成就诊**：
   - 医生点击"完成就诊"按钮，触发 `handleComplete` 方法
   - 前端向后端发送PUT请求完成就诊
   - 后端验证预约状态，更新预约状态为已就诊

### 2.2.3 关键代码讲解

1. **患者查看预约记录页面**
   【vue3/src/views/frontend/appointment/my-appointments.vue】

```
<el-timeline>
  <el-timeline-item
    v-for="appointment in filteredAppointments"
    :key="appointment.id"
    :type="getTimelineItemType(appointment.status)"
    :color="getTimelineItemColor(appointment.status)"
    :timestamp="formatDate(appointment.appointmentDate) + ' ' +
appointment.timeSlot"
    placement="top"
  >
    <el-card class="appointment-card" shadow="hover">
      <div class="appointment-info">
        <!-- 预约信息展示 -->
        <div class="appointment-actions">
          <el-button
            v-if="appointment.status === 1"
            type="danger"
            @click="handleCancelAppointment(appointment)"
            round
            plain
          >
            <el-icon><Close /></el-icon>取消预约
          </el-button>
          <el-button
            type="primary"
            @click="handleViewDetail(appointment)"
            round
          >
            <el-icon><View /></el-icon>查看详情
          </el-button>
        </div>
      </div>
    </el-card>
  </el-timeline-item>
```

```
</el-timeline>
```

2. **患者取消预约方法**
【vue3/src/views/frontend/appointment/my-appointments.vue：handleCancelAppointment方法】

```
const handleCancelAppointment = (appointment, fromDialog = false) => {
  ElMessageBox.confirm(
    '确定要取消此预约吗？取消后将无法恢复。',
    '取消预约',
    {
      confirmButtonText: '确认取消',
      cancelButtonText: '返回',
      type: 'warning'
    }
  )
    .then(async () => {
      try {
        await request.put(`/appointment/cancel/${appointment.id}`, {}, {
          successMsg: '预约已取消',
          onSuccess: () => {
            // 关闭详情对话框（如果是从详情对话框触发的）
            if (fromDialog) {
              detailDialogVisible.value = false
            }

            // 重新获取预约列表
            fetchAppointments()
          }
        })
      } catch (error) {
        console.error('取消预约失败:', error)
      }
    })
    .catch(() => {})
}
```

3. **后端取消预约方法**
【springboot/src/main/java/org/example/springboot/service/AppointmentService.java：cancelAppointment方法】

```
@Transactional
public void cancelAppointment(Long id) {
    // 检查预约是否存在
    Appointment appointment = appointmentMapper.selectById(id);
    if (appointment == null) {
        throw new ServiceException("预约不存在");
    }

    // 检查预约状态
    if (appointment.getStatus() != 1) {
        throw new ServiceException("只能取消待就诊状态的预约");
    }

    // 检查权限
```

半夜撕代码版权所有

```
    checkAppointmentPermission(appointment);

    // 更新预约状态为已取消
    appointment.setStatus(0);
    appointment.setUpdateTime(LocalDateTime.now());

    if (appointmentMapper.updateById(appointment) <= 0) {
        throw new ServiceException("取消预约失败");
    }

    // 更新排班的当前预约人数
    scheduleService.updateSchedulePatients(appointment.getScheduleId(), -1);
}
```

4. **医生/管理员预约管理页面**
【vue3/src/views/backend/appointment/index.vue】

```
<!-- 搜索表单 -->
<el-form :model="searchForm" :inline="true" class="search-form">
  <el-form-item label="患者姓名" v-if="!isDoctor">
    <el-input v-model="searchForm.patientName" placeholder="请输入患者姓名"
clearable />
  </el-form-item>
  <el-form-item label="医生姓名" v-if="!isDoctor">
    <el-input v-model="searchForm.doctorName" placeholder="请输入医生姓名"
clearable />
  </el-form-item>
  <el-form-item label="预约日期">
    <el-date-picker
      v-model="dateRange"
      type="daterange"
      range-separator="至"
      start-placeholder="开始日期"
      end-placeholder="结束日期"
    />
  </el-form-item>
  <el-form-item label="状态">
    <el-select v-model="searchForm.status" placeholder="请选择状态" clearable>
      <el-option label="待就诊" :value="1" />
      <el-option label="已就诊" :value="2" />
      <el-option label="已取消" :value="0" />
    </el-select>
  </el-form-item>
  <el-form-item>
    <el-button type="primary" @click="handleSearch">搜索</el-button>
    <el-button @click="resetSearch">重置</el-button>
  </el-form-item>
</el-form>

<!-- 预约列表 -->
<el-table v-loading="loading" :data="tableData" border style="width: 100%">
  <!-- 表格列 -->
  <el-table-column label="操作" width="200" fixed="right">
    <template #default="scope">
      <el-button type="primary" size="small"
@click="handleViewDetail(scope.row)">详情</el-button>
```

```
      <el-button
        v-if="scope.row.status === 1"
        type="success"
        size="small"
        @click="handleComplete(scope.row)"
      >
        完成就诊
      </el-button>
      <el-button
        v-if="scope.row.status === 1"
        type="danger"
        size="small"
        @click="handleCancel(scope.row)"
      >
        取消预约
      </el-button>
    </template>
  </el-table-column>
</el-table>
```

5. **医生完成就诊方法**
【vue3/src/views/backend/appointment/index.vue：handleComplete方法】

```
const handleComplete = (row, fromDialog = false) => {
  ElMessageBox.confirm(
    '确认该患者已完成就诊？',
    '完成就诊',
    {
      confirmButtonText: '确认',
      cancelButtonText: '取消',
      type: 'info'
    }
  )
    .then(async () => {
      try {
        await request.put(`/appointment/complete/${row.id}`, {}, {
          successMsg: '已标记为完成就诊',
          onSuccess: () => {
            if (fromDialog) {
              detailDialogVisible.value = false
            }
            fetchAppointments()
          }
        })
      } catch (error) {
        console.error('操作失败:', error)
      }
    })
    .catch(() => {})
}
```

6. **后端完成就诊方法**
【springboot/src/main/java/org/example/springboot/service/AppointmentService.java：completeAppointment方法】

半夜撕代码版权所有

```java
@Transactional
public void completeAppointment(Long id) {
    // 检查预约是否存在
    Appointment appointment = appointmentMapper.selectById(id);
    if (appointment == null) {
        throw new ServiceException("预约不存在");
    }

    // 检查预约状态
    if (appointment.getStatus() != 1) {
        throw new ServiceException("只能完成待就诊状态的预约");
    }

    // 检查权限（只有医生和管理员可以完成就诊）
    User currentUser = JwtTokenUtils.getCurrentUser();
    if (currentUser == null) {
        throw new ServiceException("未登录");
    }

    if (!currentUser.getRoleCode().equals("ADMIN")) {
        // 如果不是管理员，检查是否是该预约的医生
        Doctor doctor = doctorMapper.selectOne(
            new LambdaQueryWrapper<Doctor>()
                .eq(Doctor::getUserId, currentUser.getId())
        );

        if (doctor == null ||
!doctor.getId().equals(appointment.getDoctorId())) {
            throw new ServiceException("只有该预约的医生或管理员可以完成就诊");
        }
    }

    // 更新预约状态为已就诊
    appointment.setStatus(2);
    appointment.setUpdateTime(LocalDateTime.now());

    if (appointmentMapper.updateById(appointment) <= 0) {
        throw new ServiceException("完成就诊失败");
    }
}
```

### 2.2.4 功能实现细节

1. **权限控制**：
    - 患者只能查看和取消自己的预约
    - 医生只能查看和管理自己的预约
    - 管理员可以查看和管理所有预约
2. **状态管理**：
    - 预约状态包括待就诊(1)、已就诊(2)和已取消(0)
    - 只有待就诊状态的预约可以被取消或标记为已就诊
3. **排班名额管理**：
    - 取消预约时，系统会自动减少排班的当前预约人数，释放预约名额

半夜撕代码版权所有

- 完成就诊不会减少排班的当前预约人数，因为名额已经被使用

4. **用户界面优化**：
   - 使用
     我是Claude 3.7 Sonnet模型。我将继续编写预约挂号模块的讲解文档。

### 2.2.4 功能实现细节（续）

4. **用户界面优化**：
   - 使用时间轴展示患者的预约记录，直观展示预约的时间顺序
   - 不同状态的预约使用不同的颜色标识：待就诊为蓝色，已就诊为绿色，已取消为灰色
   - 预约详情对话框展示完整的预约信息，包括患者信息、医生信息、预约时间等
   - 后台管理界面支持多条件筛选，方便医生和管理员快速查找预约记录
5. **数据验证**：
   - 前端表单验证确保症状描述等必填字段不为空
   - 后端验证确保预约的合法性，包括检查患者、医生和排班是否存在
   - 系统会检查排班是否已满，避免超出最大接诊人数

## 2.3 与其他模块的交互

### 2.3.1 与排班模块的交互

预约挂号模块需要从排班模块获取医生的排班信息，包括排班日期、时间段和可预约名额。当患者成功预约或取消预约时，系统会更新排班的当前预约人数。

```java
// 更新排班的当前预约人数
@Transactional
public void updateSchedulePatients(Long scheduleId, int increment) {
    Schedule schedule = scheduleMapper.selectById(scheduleId);
    if (schedule == null) {
        throw new ServiceException("排班不存在");
    }

    // 更新当前预约人数
    Integer currentPatients = schedule.getCurrentPatients();
    if (currentPatients == null) {
        currentPatients = 0;
    }

    currentPatients += increment;

    // 确保不小于0
    if (currentPatients < 0) {
        currentPatients = 0;
    }

    // 确保不超过最大接诊人数
    if (currentPatients > schedule.getMaxPatients()) {
        throw new ServiceException("超出最大接诊人数");
    }

    schedule.setCurrentPatients(currentPatients);
```

半夜撕代码版权所有

```
    schedule.setUpdateTime(LocalDateTime.now());

    if (scheduleMapper.updateById(schedule) <= 0) {
        throw new ServiceException("更新排班信息失败");
    }
}
```

### 2.3.2 与就诊记录模块的交互

当医生完成就诊后，预约状态会更新为已就诊，并可以创建就诊记录。就诊记录会关联预约ID，建立预约与就诊记录的关联关系。

```
// 就诊记录中关联预约ID
@Transactional
public MedicalRecord createMedicalRecord(MedicalRecord medicalRecord) {
    // 如果有关联预约，检查预约是否存在
    if (medicalRecord.getAppointmentId() != null) {
        Appointment appointment =
appointmentMapper.selectById(medicalRecord.getAppointmentId());
        if (appointment == null) {
            throw new ServiceException("预约不存在");
        }

        // 更新预约状态为已就诊
        if (appointment.getStatus() == 1) {
            appointmentService.completeAppointment(appointment.getId());
        }
    }

    // 其他创建就诊记录的逻辑...
}
```

### 2.3.3 与患者模块的交互

预约挂号前，系统会检查用户是否已完善患者信息。如未完善，则提示用户先完善患者信息再进行预约。预约时会关联患者ID，建立预约与患者的关联关系。

```
// 检查是否有患者信息
if (!userStore.patientInfo) {
  ElMessageBox.confirm(
    '您尚未完善患者信息，需要先完善患者信息才能预约',
    '提示',
    {
      confirmButtonText: '去完善',
      cancelButtonText: '取消',
      type: 'warning'
    }
  )
    .then(() => {
      router.push('/profile')
    })
    .catch(() => {})
  return
}
```

# 三、总结

## 3.1 预约挂号功能实现

预约挂号功能通过前端的分步选择流程实现，用户首先选择科室，然后选择医生，最后选择预约时间。整个流程中，前端通过`fetchDepartments`、`fetchDoctorsByDepartment`和`fetchDoctorSchedules`方法依次获取科室列表、医生列表和排班信息。用户选择排班后，通过`handleAppointment`方法打开预约确认对话框，填写症状描述后提交预约。后端的`createAppointment`方法负责验证预约信息，包括检查患者和医生是否存在、排班是否有剩余名额等，验证通过后创建预约记录并更新排班的当前预约人数。整个预约流程设计合理，用户体验良好，确保患者能够方便地完成预约挂号。

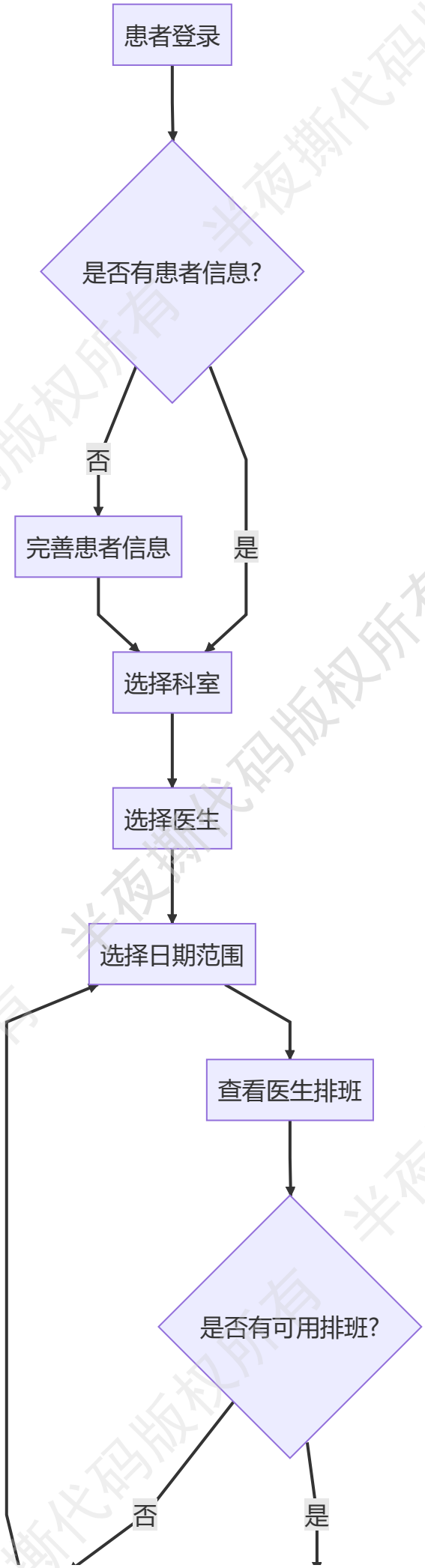
## 3.2 预约管理功能实现

预约管理功能分为患者端和管理端两部分。患者端通过时间轴形式展示个人预约记录，支持查看详情和取消预约。管理端提供多条件筛选功能，方便医生和管理员查找和管理预约记录。取消预约时，系统会通过`cancelAppointment`方法更新预约状态并释放排班名额。完成就诊时，系统会通过`completeAppointment`方法更新预约状态为已就诊。整个预约管理功能实现了严格的权限控制，确保数据安全和业务流程的正确执行。

半夜撕代码版权所有

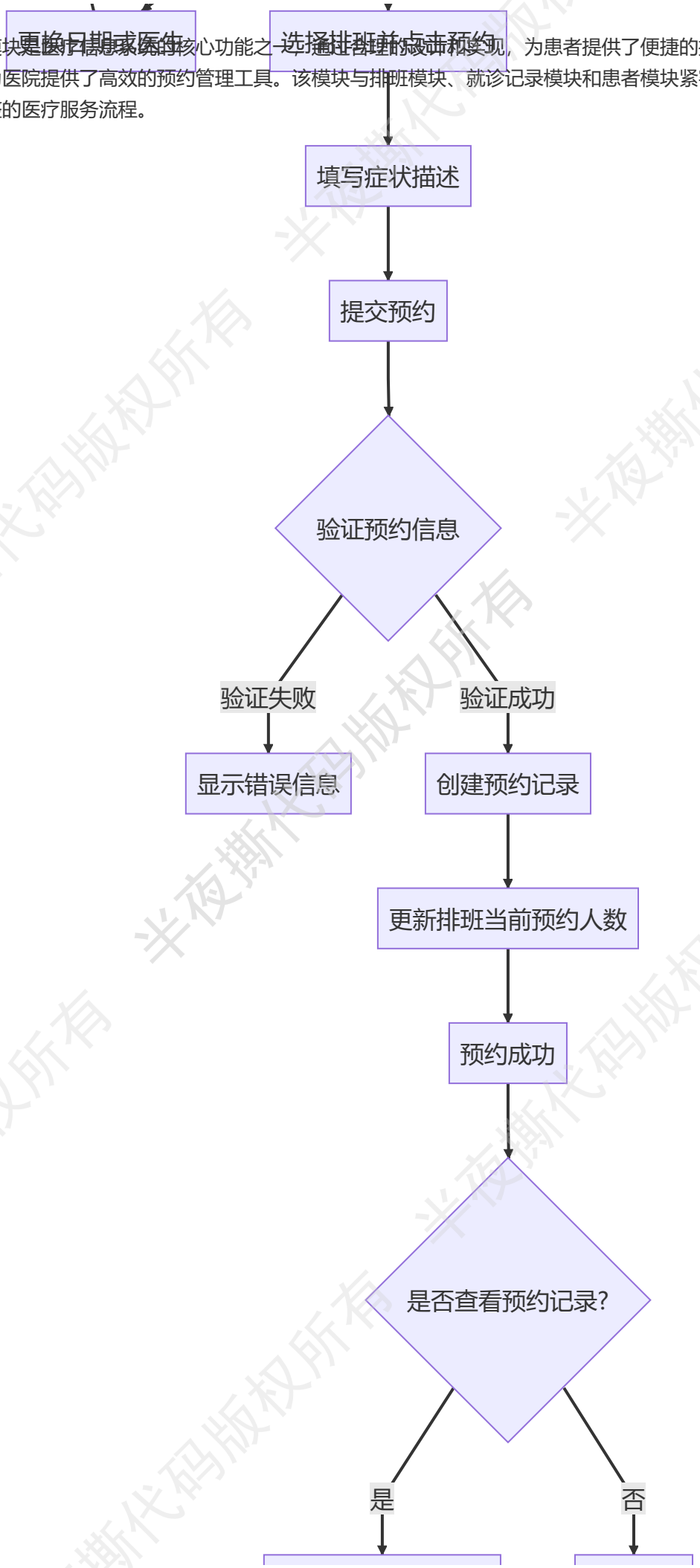
### 3.3 预约挂号流程图

半夜撕代码版权所有




半夜撕代码版权所有

预约挂号模块是医疗信息系统的核心功能之一，通过合理的设计和实现，为患者提供了便捷的挂号服务，同时为医院提供了高效的预约管理工具。该模块与排班模块、就诊记录模块和患者模块紧密结合，形成了完整的医疗服务流程。




半夜撕代码版权所有

跳转到我的预约页面

继续预约

半夜撕代码版权所有